CS 182 - HW 6 - Jie Wen

1. Memory considerations when using GPUs (Coding Question)

(a) Managing GPU memory for training neural networks (Notebook Section 1).

(i)

`Model parameters` : 58164298
`Model size estimate (MB)` : 232.657192

(ii)

Based on the `nvidia-smi` output:

- The GPU being used is a **Tesla T4**.
- It has a total of **15360MiB** of memory.

(iii)

Following the hint `(total GPU memory utilization - model size)`:

- Total GPU memory utilization from `nvidia-smi` : **346 MiB**
- Model size from the first output: **232.66 MB**
- CUDA Context Overhead ≈ 346 MB - 232.66 MB = **113.34 MB**

(b) Optimizer memory usage (Notebook Section 2).

(i)

==== Memory Profiling Results ====

`SGD` : 2172.0 MB
`SGD_WITH_MOMENTUM` : 2254.0 MB
`ADAM` : 2418.0 MB

(ii)

The **Adam optimizer** consumes the most GPU memory (2418 MB).

This is because Adam maintains **two additional parameter states** — the **first moment (mean of gradients)** and the **second moment (uncentered variance)** — for every trainable parameter in the model.

By contrast:

- **SGD** only stores gradients (1 tensor per parameter).
- **SGD with Momentum** adds **one extra tensor** (the momentum buffer).
- **Adam** adds **two** such buffers (m and v), which roughly **triples the memory needed per parameter** compared to plain SGD.

Therefore, the Adam optimizer's additional state tracking leads to higher GPU memory usage.

(c) Batch size, learning rates and memory utilization (Notebook Section 3)

(i)

As the batch size increases, GPU memory usage rises almost linearly because larger batches require storing more activations and gradients.

The largest batch size you were able to train successfully on your GPU (Tesla T4 ≈ 15 GB) is **256**, which used ≈ 7.75 GB of memory and did not trigger an out-of-memory error.

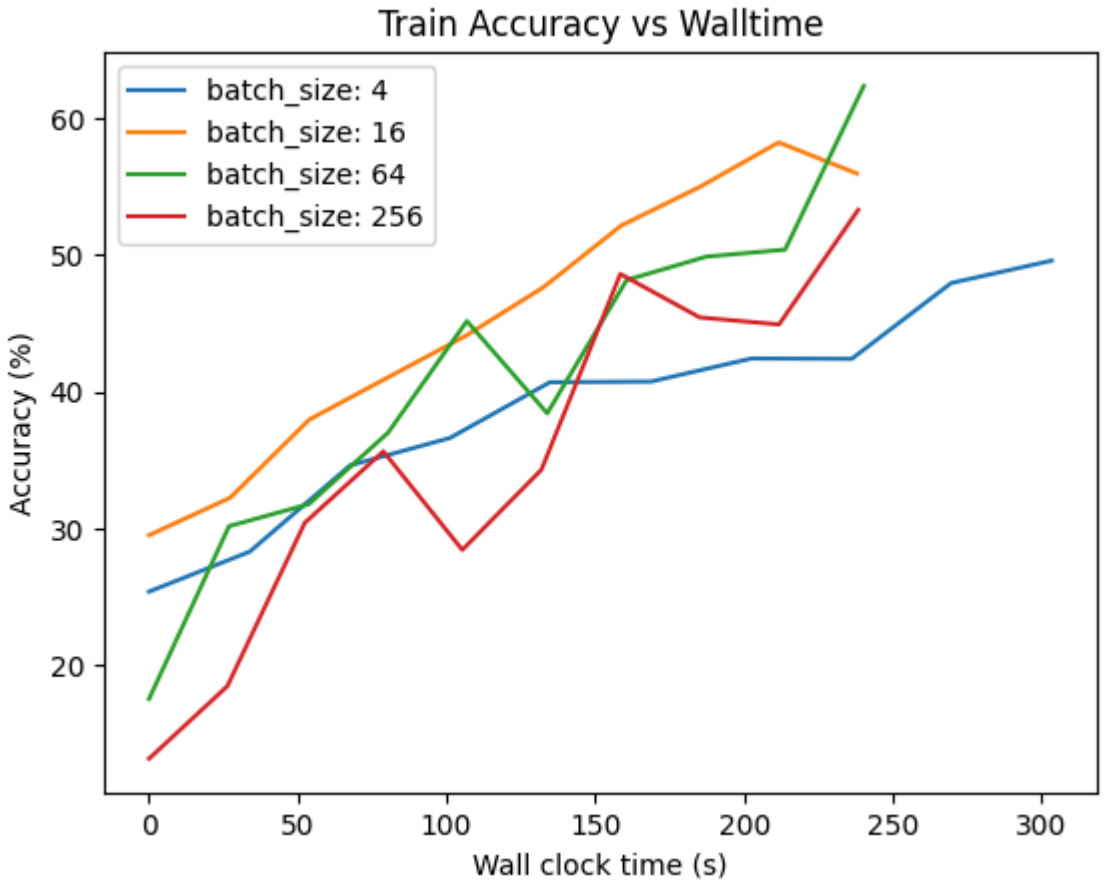
(ii)

The **batch size 64** model reached the **highest training accuracy (~62 %)** after 10 epochs.

(iii)

The **batch size 256** run completed 10 epochs the **fastest** because larger batches perform **fewer gradient-update steps per epoch** and keep the GPU more fully utilized, improving throughput.

(iv)



---

2. Graph Dynamics and GNN Concepts

(a) Write the output of the j-th node at layer k in this network in terms of the matrix A.

In this linear GNN, each layer's computation is equivalent to a multiplication by the adjacency matrix A. The input to the 0-th layer for node j is a one-hot vector, denoted as e_j. After k layers, the output is the result of applying the matrix A for k times.

The output of node j at layer k, which is an n-dimensional vector, is given by:

$$\text{Output}^{(k)}(j) = A^k e_j$$

This expression represents the j-th column of the matrix A^k. The i-th component of this output vector is therefore the (i, j)-th entry of the matrix A^k.

For example, at layer $k = 1$, the output for node j is $\text{Output}^{(1)}(j) = Ae_j$. The i-th component of this vector is $A_{i,j}$, which is 1 if node i is a neighbor of node j, and 0 otherwise. At layer $k = 2$, the output is $\text{Output}^{(2)}(j) = A^2 e_j$, and so on.

(b) Prove that the i-th output of node j at layer k in the network above is the count of how many paths there are from i to j of length k.

Let $L_k(i, j)$ be the number of distinct paths of length exactly k from vertex i to vertex j. The proof that the i-th component of the output of node j at layer k, denoted $\text{Output}^{(k)}(j)_i$, is equal to $L_k(i, j)$ proceeds by induction on k.

Base Case ($k = 0$):
A path of length 0 from node i to node j exists only if $i = j$. By convention, $L_0(i, j) = 1$ if $i = j$ and 0 otherwise. The input to the network at layer 0 for node j is the one-hot vector e_j. The i-th component of this vector is $\text{Output}^{(0)}(j)_i$, which is 1 if $i = j$ and 0 otherwise. This matches $L_0(i, j)$, so the base case holds.

Inductive Step:
Assume for some $k \geq 0$ that $\text{Output}^{(k)}(j)_i = L_k(i, j)$ for all nodes i, j.
The output at layer $k + 1$ is calculated by summing the weighted inputs from the previous layer. For node j, the i-th component of its output at layer $k + 1$ is:

$$\text{Output}^{(k+1)}(j)_i = \sum_{r \in V(j)} \text{Output}^{(k)}(r)_i$$

where $V(j)$ is the set of neighbors of node j.

By the inductive hypothesis, $\text{Output}^{(k)}(r)_i = L_k(i, r)$. Substituting this into the equation gives:

$$\text{Output}^{(k+1)}(j)_i = \sum_{r \in V(j)} L_k(i, r)$$

A path of length $k+1$ from i to j consists of a path of length k from i to a neighbor of j, say r, followed by the edge from r to j. The term $L_k(i, r)$ counts the number of paths of length k from i to r. Summing over all neighbors r of j counts every distinct path of length $k+1$ from i to j exactly once.
Therefore, the sum is equal to $L_{k+1}(i, j)$. This shows that $\text{Output}^{(k+1)}(j)_i = L_{k+1}(i, j)$, completing the induction.

(c) What is the update function? In the case of the computations in previous parts, what is the update function that takes the aggregated inputs from neighbors and results in the output for this node?

If the activations of all nodes at layer k are represented by a matrix $H^{(k)}$ (where the j-th row is the feature vector for node j), the aggregation step can be written in matrix form. The aggregated information from neighbors, $Z^{(k)}$, is computed as:

$$Z^{(k)} = AH^{(k)}$$

In this specific linear network, the update function is the identity function. It simply passes the aggregated vector to the next layer without any non-linear activation or transformation.

$$H^{(k+1)} = Z^{(k)}$$

The overall update function for a full layer's computation is therefore $H^{(k+1)} = AH^{(k)}$. Given that the initial input is the identity matrix $H^{(0)} = I$ (where each row is a one-hot vector), this confirms that $H^{(k)} = A^k$.

(d) What is the interpretation of the outputs of node j at layer k if sum aggregation is replaced with max aggregation?

If the `sum` aggregator is replaced with a `max` aggregator, the output's interpretation changes from counting paths to indicating the existence of paths. The output becomes a binary indicator of reachability.

At layer 0, node j starts with a one-hot vector e_j. At layer 1, the i-th component of node j 's output vector will be 1 if $\max_{r \in V(j)}(\text{Output}^{(0)}(r)_i) = 1$. This occurs if and only if there is a neighbor r of j such that $r = i$. In other words, the output is 1 if there is a direct edge (a path of length 1) from i to j.

By induction, at layer k, the i-th component of node j 's output vector will be 1 if and only if there exists at least one path of length k from node i to node j. The `max` aggregator acts like a logical OR, propagating the existence of a path without counting how many such paths exist.

(e) You are studying how organic molecules break down when heated.

(i) How would you represent this as a graph?

The molecule can be represented as a graph with the following components:

- **Nodes**: Each atom in the molecule is a node. Node features would include the atom's element type (e.g., carbon, oxygen), encoded perhaps as a one-hot vector or a learned embedding.
- **Edges**: Each chemical bond between two atoms is an edge. Edge features can include bond properties like bond length and bond type (e.g., single, double).
- **Global State**: A global feature vector can represent properties of the entire molecule, such as the molecule type (e.g., carbohydrate, protein).
- **Directionality**: The graph should be treated as **undirected**, as chemical bonds are mutual connections allowing influence to flow in both directions between atoms.

(ii) How would you use the outputs of the last GNN layer to make the prediction?

The task is to predict which bond will break first, which is an edge-level prediction problem. After several layers of message passing, the GNN will produce final embeddings for each edge.

1. A prediction head, such as a small multi-layer perceptron (MLP) or a single linear layer, is applied to each final edge embedding.
2. This head outputs a score or probability for each bond, indicating its likelihood of breaking first.
3. The bond with the highest score is the model's prediction for the first bond to break. This can be found using an `argmax` operation over all edge scores in the graph.

(iii) How would you encode the node representation for the input to the GNN?

The input node representation for each atom should be a feature vector. A common way to encode the element type is using a **one-hot encoding**. If there are N possible elements in the dataset, each atom's feature vector would be an n-dimensional vector with a 1 in the position corresponding to its element and 0s elsewhere.

Alternatively, a learnable embedding could be used, where each element type is mapped to a dense vector whose values are learned during training. Other atomic properties, if available (such as atomic mass or valence), could be included in the feature vector, typically after normalization.

(f) Fill out the rest of the table.

CNN	GNN
Image classification	Graph-level prediction problem
Pixel-level classification / Image segmentation	Node-level prediction problem

CNN	GNN
Color jitter data augmentation (adjusting color/brightness)	Node/edge feature augmentation (randomly perturbing node or edge features to increase robustness)
Image flip data augmentation	Graph structure augmentation (e.g., for geometric graphs, applying rotations/reflections; for general graphs, this has no direct equivalent due to permutation invariance)
Channel dropout	Feature dropout (randomly setting entire dimensions of node/edge feature vectors to zero across the graph)
Zero padding edges	Adding self-loops (connecting each node to itself to ensure its own features are included in updates) or, for batching, padding smaller graphs with dummy nodes/edges to create uniform tensors.
ResNet skip connections	Skip connections in GNNs (adding or concatenating a node's representation from a previous layer to a later layer to combat over-smoothing and ease training of deep GNNs)
Blurring an image	Graph smoothing / feature propagation (averaging a node's features with its neighbors, which is an inherent part of many GNN layers; repeated application leads to feature homogenization)
Image inpainting (predicting missing pixels)	Predicting missing values of nodes (using the GNN to infer missing node features or labels from the features and structure of the surrounding graph)

(g) How would you use this graph for prediction if you're doing a graph-level classification problem, but node values are missing for some of your graph nodes?

For a graph-level classification task with missing node features, the graph can still be used effectively.

1. **Impute or Initialize Missing Features**: Nodes with missing features can be initialized with a placeholder value. Common strategies include using a vector of zeros, the mean feature vector of the known nodes, or a separate, learnable embedding that represents "missing." An additional binary feature can be added to all nodes to indicate whether their original features were present or missing.
2. **Rely on Message Passing**: The core strength of GNNs is message passing. Even if a node starts with a zero vector, it will aggregate information from its neighbors over several GNN layers. After a few iterations, its representation will be a function of its local neighborhood's structure and features, effectively imputing its state based on context.
3. **Proceed with Graph-Level Prediction**: The entire graph, including the imputed nodes, is processed by the GNN. The final graph-level representation is obtained through a pooling mechanism (e.g., averaging all node embeddings). This pooled representation is then fed into a classifier to make the final prediction. The model learns to make predictions using the available structural and feature information, implicitly handling the imputed values.

(h) Consider the graph neural net architecture shown in Figure 2.

(i) How does doubling the number of nodes in a graph change the number of learned weights and the amount of computation?

- **Number of Learned Weights**: The number of learned weights **does not change**. GNNs apply the same set of learned functions (with shared weights) to every node, edge, and the global state. The model's parameters are independent of the input graph's size.
- **Amount of Computation (if average node degree remains the same)**: The computation will roughly **double**. GNN computation scales approximately linearly with the number of edges. If the number of nodes doubles and the average degree is constant, the number of edges also doubles, leading to a twofold increase in computation.
- **Amount of Computation (if the graph is fully connected)**: The computation will roughly **quadruple**. In a fully connected graph with n nodes, the number of edges is proportional to n^2. If the number of nodes doubles to $2n$, the number of edges becomes proportional to $(2n)^2 = 4n^2$. Thus, the computation increases by a factor of approximately four.

(ii) Where in this network are learned weights/parameters?

Learned weights (parameters) are located within the update functions that are applied at each step of the GNN. Specifically, they are in the neural networks (e.g., MLPs) used for:

- The **edge update function**, which computes new edge embeddings.
- The **node update function**, which computes new node embeddings.
- The **global state update function**, which computes a new global embedding.

These sets of weights are shared across all elements of the same type (all nodes share one set of node-update weights, etc.) and do not grow with the size of the graph.

(iii) The diagram provided shows undirected edges. How would you incorporate directed edges?

To incorporate directed edges, the message-passing mechanism must be modified to respect the direction of connections.

- **Asymmetric Aggregation**: When updating a node's features, the aggregation step should only consider messages from its **incoming** neighbors (predecessors). An edge from node i to node j means that node i sends a message to node j, but not vice versa (unless a reverse edge from j to i also exists).
- **Implementation**: This is typically implemented by using a directed adjacency matrix or by constructing separate adjacency lists for incoming and outgoing edges for each node. The aggregation function would then iterate over the list of predecessors for each node. The core GNN architecture remains the same, but the neighborhood definition used for aggregation becomes directed. An undirected edge can be modeled as two reciprocal directed edges.

---

3. Graph Neural Networks

(a) Valid vs. invalid update forms

The key requirement is that since neighbors of i form an **unordered set** in an undirected, unlabeled graph, the aggregation over $m^{\ell-1} * i, j * j = 1^{n_i}$ must be **permutation-invariant** (e.g., sum/mean/max with shared parameters).

(i) $s_i^\ell = w_1 s_i^{\ell-1} + w_2 \frac{1}{n_i} \sum_{j=1}^{n_i} m_{i,j}^{\ell-1}$
Valid.
It uses a mean (symmetric) aggregator; parameters w_1, w_2 are shared and independent of neighbor order. This is the standard "message passing + (normalized) neighborhood average" pattern.

(ii) $s_i^\ell = \max(w_1 s_i^{\ell-1}, w_2 m_{i,1}^{\ell-1}, \ldots, w_{n_i+1} m_{i,n_i}^{\ell-1})$ (component-wise max)
Invalid.
It assigns **different weights per neighbor index** $(w_2, \ldots, w_{n_i+1})$, which **depends on an arbitrary ordering** of neighbors and on n_i; that breaks permutation invariance and parameter sharing across nodes of varying degree.

(iii) $s_i^\ell = \max(w_1 s_i^{\ell-1}, w_2 m_{i,1}^{\ell-1}, \ldots, w_2 m_{i,n_i}^{\ell-1})$ (component-wise max)
Valid.
Max is a symmetric set function, and using a **shared** w_2 across all neighbors preserves permutation invariance and works for variable n_i. (Max/mean/sum with shared transforms are canonical invariant set aggregators.)

(b) Training loss on the given split

Held-out nodes: **1 and 4**. Training nodes: **2, 3, 5** with labels $y_2 = 1, y_3 = 1, y_5 = 0$ and predictions $\hat{y}_2 = b, \hat{y}_3 = c, \hat{y}_5 = e$.

Average binary cross-entropy over the training set ($n = 3$):
$\mathcal{L}_{\text{train}} = \frac{1}{3}\Big(y_2 \log \frac{1}{b} + y_3 \log \frac{1}{c} + (1 - y_5) \log \frac{1}{1-e}\Big) = \frac{1}{3}\Big(- \log b - \log c - \log(1 - e)\Big)$.

(c) Properties and dimensions for the update

$\mathbf{s}_i^\ell = \mathbf{s}_i^{\ell-1} + W_1 \frac{1}{n_i} \sum_{j=1}^{n_i} \tanh\left(W_2 \mathbf{m}_{i,j}^{\ell-1}\right)$ (1)

(i) Design patterns

- **Residual connection: Yes.** The skip term $\mathbf{s}_i^{\ell-1} \to \mathbf{s}_i^\ell$ is exactly a residual/skip connection.
- **Batch normalization: No.** There is no normalization term (BN) in (1). (BN would add a per-batch affine normalization layer.)

(ii) Dimensions of W_1

Let $\mathbf{s} \in \mathbb{R}^d$. If W_2 has k rows, then $W_2\mathbf{m} \in \mathbb{R}^k$, the mean over neighbors stays in $\mathbb{R}^k$, and W_1 must map $\mathbb{R}^k \to \mathbb{R}^d$.
Answer: $W_1 \in \mathbb{R}^{d \times k}$.

(iii) Expanded updates on the given graph

Take $\mathbf{m}_{i,j}^{\ell-1} = \mathbf{s}_{\text{neighbor}(i,j)}^{\ell-1}$.

From Figure 3, neighbors are:

- $\mathcal{N}(2) = 1, 4, 5 \Rightarrow n_2 = 3$
- $\mathcal{N}(3) = 5 \Rightarrow n_3 = 1$

So:

$$\mathbf{s}_2^\ell = \mathbf{s}_2^{\ell-1} + W_1 \frac{1}{3}\Big[\tanh\left(W_2 \mathbf{s}_1^{\ell-1}\right) + \tanh\left(W_2 \mathbf{s}_4^{\ell-1}\right) + \tanh\left(W_2 \mathbf{s}_5^{\ell-1}\right)\Big]$$

$$\mathbf{s}_3^\ell = \mathbf{s}_3^{\ell-1} + W_1 \tanh\left(W_2 \mathbf{s}_5^{\ell-1}\right)$$

---

4. Exploring Deep Learning Tooling

(a)

What is easy about TensorBoard?

- **Drop-in logging & instant visuals.** With `SummaryWriter.add_scalar`, time-series plots for loss/accuracy can be generated with almost no code. The UI supports smoothing and quick comparisons across runs.
- **Multiple dashboards out of the box.** Scalars, Images, Histograms, Graphs, Embeddings, HParams, and Profiler are all first-class features.
- **Simple run organization.** If each run is placed in its own subfolder and `--logdir` points to the parent, the UI overlays/compares them automatically.

What are the drawbacks?

- **Run & metadata management is manual.** Folder names must be hand-crafted, old logs pruned, and care taken to avoid reusing a logdir (which merges curves). HParams works, but it is more TF-centric and can be finicky in PyTorch if not logged exactly right.
- **Performance degrades with many/large logs.** With hundreds of runs or large event files (hundreds of MB+), loading and UI interactions can get slow or memory-hungry.
- **Sharing & experiment lifecycle.** There is no built-in cloud history/versioning or per-experiment "entity/project" concept; collaboration beyond a single machine requires extra setup.

Would it still be easy with *massive* numbers of experiments?

- **Usable with curation, but not ideal at scale.** If (a) strict run naming and one-run-per-folder is enforced, (b) old event files are archived/rotated, and (c) scalar logging frequency is kept reasonable, TensorBoard remains fine for dozens of runs. When scaling to hundreds/thousands of runs and very large event logs, start-up and navigation get sluggish.

How organized is it?

- **Good, if a disciplined approach is taken.** TensorBoard will neatly group and compare runs if a consistent directory layout and tags are used. The recommended pattern is:

  ```
  runs/
    convnet_bs64_lr1e-3_SGD/
    convnet_bs64_lr3e-4_AdamW/
    resnet18_bs128_lr1e-3_SGD/
    ...
  ```

  Then launch with `tensorboard --logdir runs`. An HParams summary can be added so the **HPARAMS** tab shows a sortable table (final val acc/loss vs. hyperparams).

(b)

Weights & Biases (W&B) provides several features that **TensorBoard** either lacks or only partially supports:

- **Built-in, cloud-hosted experiment hub** with projects/workspaces that can be opened from any machine and shared via links (no ssh tunnels / file copies). It is also possible to mirror TensorBoard logs into W&B.
- **Automated hyperparameter sweeps** (random/grid/Bayesian), with orchestration across many machines/agents, and first-class dashboards for comparing runs. TensorBoard's HParams allows for the *visualization* of trials, but it does not *run* or coordinate them.
- **Artifacts (dataset/model/versioning + lineage)** for tracking exactly which data/model/checkpoints each run used/produced; this creates a reproducible pipeline graph that TensorBoard lacks.
- **Tables** (interactive, row-level analysis of examples/metrics) for slicing, filtering, and comparing predictions across runs—beyond TensorBoard's scalar/PR/embedding views.
- **Centralized run metadata & reproducibility**: auto-capture of configs, code/version, system stats, and easy cross-run comparisons & reports for collaboration.
- **Reporting & sharing**: The ability to publish W&B Reports that bundle runs, charts, and commentary for teammates/stakeholders—no extra hosting needed.

Bottom line: TensorBoard is a tool for local visualization (scalars, images, graphs, HParams), whereas W&B adds hosted collaboration, automated sweeps, dataset/model versioning, and richer run comparison/analysis tooling designed for managing *many* experiments at once.

Zachary's Karate Club

Zachary's karate club (ZKC) is a social network of a university karate club, described in the paper "An Information Flow Model for Conflict and Fission in Small Groups" by Wayne W. Zachary.

The social network captures 34 members of a karate club, documenting links between pairs of members who interacted outside the club.

During the study, a conflict arose between the officer/ administrator ("John A") and the instructor "Mr. Hi", which led to the split of the club into two.

Part of the members formed a new club around Mr. Hi; and the remaining members went with the officer.

Based on collected data Zachary correctly assigned all but one member of the club to the groups they actually joined after the split. You could read more about it here https://en.wikipedia.org/wiki/Zachary%27s_karate_club, here https://www.jstor.org/stable/3629752, and here https://commons.wikimedia.org/wiki/File:Social_Network_Model_of_Relationships_in_the_Karate_Club.png

Import Dependencies

```
import numpy as np
import networkx as ntwx
from scipy.linalg import sqrtm
import matplotlib.pyplot as plt
from matplotlib import animation
from IPython.display import HTML as web
from networkx.algorithms.community.modularity_max import
greedy_modularity_communities

%matplotlib inline

def set_seed(seed=100):
    """sets seed"""
    np.random.seed(seed)
```

Processing Helper Functions

```
def get_graph_metadata(graph, key=None):
    """Return metadata obout a graph i.e. number_of_nodes,
number_of_edges and node attributes"""
    if key is None:
        return graph.number_of_nodes(), graph.number_of_edges()
    return graph.number_of_nodes(), graph.number_of_edges(),
ntwx.get_node_attributes(graph, key)
```

```
def get_xavier_init(input_dim, output_dim):
    """returns xaviers in initializer"""
    std_dev = np.sqrt(5.0 / (input_dim + output_dim))
    return np.random.uniform(-std_dev, std_dev, size=(input_dim,
output_dim))

def get_cross_entropy(pred, labels):
    """computes crossentropy between the predictions and the labels"""
    return -np.log(pred)[np.arange(pred.shape[0]), np.argmax(labels,
axis=1)]


def normalized_difference_norm(dW, dW_approx):
    """compares the deirivarive of the weight with its apprimation"""
    return np.linalg.norm(dW - dW_approx) / (np.linalg.norm(dW) +
np.linalg.norm(dW_approx))

def get_colors_labels_and_classes(graph, num_nodes):
    """We applied greedy modurality maximization from the original
    paper https://arxiv.org/pdf/1609.02907.pdf to come up with cluster
    labels for each of the members of the club. We will train our GNN
    to predict these cluster labels."""
    clusters = greedy_modularity_communities(graph)
    unsure_cluster = clusters[-1]
    clusters = clusters[:2]
    clusters[1] = clusters[1].union(unsure_cluster)
    color_lists = np.zeros(num_nodes)
    for i, cluster in enumerate(clusters):
        color_lists[list(cluster)] = i
    classes = np.unique(color_lists).shape[0]
    return color_lists, np.eye(classes)[color_lists.astype(int)],
classes

def get_affiliation(club_labels):
    """return the affiation of the karate club members"""
    Mr_Hi, Officer = [], []
    for key, value in club_labels.items():
        if value == 'Mr. Hi':
            Mr_Hi.append(key)
        else:
            Officer.append(key)
    return Mr_Hi, Officer


def fill_diagonal(source_array, diagonal):
    """helps fill element of the source array into a diagonal
matrix"""
    copy = source_array.copy()
```

```
    np.fill_diagonal(copy, diagonal)
    return copy
```

The original paper on greedy modularity communities maximization could be found here https://journals.aps.org/pre/pdf/10.1103/PhysRevE.70.066111?casa_token=Fqnjw_t-J64AAAAA%3ADmyzj146CDE-UeW_1I6Ifvu40GmCC_goDC4i6lvkYIa9GENKcktHxOgHO5et7Z7xJ3NU1q2Ngt2J6Zs

Visualization Helper Function

```
def show_graph(graph,
               label_values_of_nodes,
               label_colors_of_nodes,
               colors_of_edges='black',
               display_window_size=15,
               positions_of_nodes=None,
               cmap='jet'):
    """helps visualize the graph"""
    fig, ax = plt.subplots(figsize=(display_window_size,
display_window_size))
    if positions_of_nodes is None:
        #
https://networkx.org/documentation/stable/reference/generated/networkx
.drawing.layout.spring_layout.html
        positions_of_nodes = ntwx.spring_layout(graph,

k=5/np.sqrt(graph.number_of_nodes()))
        #
https://networkx.org/documentation/stable/reference/drawing.html
    ntwx.draw(
        graph,
        positions_of_nodes,
        with_labels=label_values_of_nodes,
        labels=label_values_of_nodes,
        node_color=label_colors_of_nodes,
        ax=ax,
    cmap=cmap,
    edge_color=colors_of_edges)


def plot_training_curves(train_losses, test_losses, grid=False):
    """shows training curves"""
    fig = plt.figure(figsize=(10, 4))
    ax1 = fig.add_subplot(121)
    ax1.plot(np.log10(train_losses), label='train')
    ax1.plot(np.log10(test_losses), label='test')
    ax1.legend()
    if grid:
        ax1.grid()
```

```
    ax2 = fig.add_subplot(122)
    ax2.plot(accs, label='acc')
    ax2.set(ylim=[0,1])
    ax2.legend()

    if grid:
        ax2.grid()
```

Gradient Update Helper Function

```
def compute_gradients(param_name, layer, inputs_data, gt_labels,
eps=1e-4, weight_decay=0):
    """Compute the gradient with respect to a a given parameter in a
given layer"""
    gradients_utils = {}
    batch_size = gt_labels.shape[0]
    replica = getattr(layer, param_name).copy()
    replica_flattened = np.asarray(replica).flatten()
    gradients_utils['gradient_values'] =
np.zeros(replica_flattened.shape)
    n_parms = replica_flattened.shape[0]
    for ind, param in enumerate(replica_flattened):
        # lower bound cost
        replica_flattened[ind] = param - eps
        temp = replica_flattened.reshape(replica.shape)
        gradients_utils['lower_bound_pred'] =
layer.forward_pass(*inputs_data, **{param_name: temp})
        decay = weight_decay/ 2 * np.sum(replica_flattened ** 2) /
batch_size
        lower_cost =
np.mean(get_cross_entropy(gradients_utils['lower_bound_pred'],
                                                gt_labels)) + decay
        # upper bound cost
        replica_flattened[ind] = param + eps
        temp = replica_flattened.reshape(replica.shape)
        gradients_utils['upper_bound_pred'] =
layer.forward_pass(*inputs_data, **{param_name: temp})
        decay = weight_decay/ 2 * np.sum(replica_flattened**2) /
batch_size

        upper_cost =
np.mean(get_cross_entropy(gradients_utils['upper_bound_pred'],
                                                gt_labels)) + decay
        gradients_utils['gradient_values'][ind] = ((upper_cost -
lower_cost) / (2 * eps))
        replica_flattened[ind] = param
    return gradients_utils['gradient_values'].reshape(replica.shape)
```

Grad Descent Optimizer Helper Function

```
class Grad_Descent_Optimizer():
    """Performs Gradient Descent"""
    def __init__(self, learning_rate, weight_decay):
        self.learning_rate = learning_rate
        self.weight_decay = weight_decay
        self._y_pred = None
        self._y_true = None
        self._output = None
        self.batch_size = None
        self.nodes_to_be_trained = None

    def __call__(self, y_pred, y_true,
nodes_to_be_used_for_training=None):
        self.y_pred = y_pred
        self.y_true = y_true
        self.batch_size = y_pred.shape[0]

        if nodes_to_be_used_for_training is None:
            self.nodes_to_be_used_for_training =
np.arange(self.batch_size)
        else:
            self.nodes_to_be_used_for_training =
nodes_to_be_used_for_training

    @property
    def out(self, ):
        return self._output

    @out.setter
    def out(self, y):
        self._output = y
```

Set seed

```
set_seed()
```

Implementation Check Helper Function

```
def implementation_check(dW, dW_approx, db, db_approx):
    """This function helps check how correct in your impplementation a
layer"""
    try:
        assert normalized_difference_norm(dW, dW_approx) < 1e-7
        assert normalized_difference_norm(db, db_approx) < 1e-7
        print('congrats, your implementation passes the test !!!')
    except:
        print('Not quite there :( yet; your implementation did not
pass the test')
```

Training Helper Function

```
def train_test_split(test_nodes, labels):
    return np.array([i for i in range(labels.shape[0]) if i not in
test_nodes])

def threshold(arr, threshold_value=0.5):
    """This function treshold the output of a softmax to either be 0
or 1"""
    arr[arr>=threshold_value]=1
    arr[arr<threshold_value]=0
    return arr
```

Node Classification Helper Function

```
class Softmax_Layer():
    """applies a weight multiplication and returns the forward and
backward passes softmax. Some values are cached
    so we could use them to compute the gredients.
    Returns: (batch_size, output_dim)"""
    def __init__(self, input_dim, output_dim, name=''):
        self.name = name
        self.cache = {}
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.bias = np.zeros((self.output_dim, 1))
        self.W = get_xavier_init(self.output_dim, self.input_dim)

    def __repr__(self):
        dims = (self.input_dim, self.output_dim)
        if self.name:
            return f"Softmax_Layer: W{'_' + self.name}{dims}"
        else:
            return f"Softmax_Layer: W{'_'+ ''}{dims}"

    def get_softmax(self, x):
        x = x - np.max(x, axis=0, keepdims=True)
        exp_x = np.exp(x)
        return exp_x / np.sum(exp_x, axis=0, keepdims=True)

    def forward_pass(self, X, W=None, bias=None):
        """Returns the softmax of the input X after appliying the
weight and biases."""
        self.cache['X'] = X.T
        if W is None:
            W = self.W
        if bias is None:
            bias = self.bias

        return self.get_softmax(np.asarray(W @ self.cache['X']) +
bias).T # (batch_size, output_dim)
```

```
    def backward_pass(self, optimizer, need_update=True):
        """mask nodes nodes not revelant for training, and updates
optimizer parameters"""
        training_mask = np.zeros(optimizer.y_pred.shape[0])
        training_mask[optimizer.nodes_to_be_trained] = 1
        training_mask = training_mask.reshape((-1, 1))

        dLoss = np.asarray((optimizer.y_pred - optimizer.y_true))
        dLoss = np.multiply(dLoss, training_mask)

        self.grad = dLoss @ self.W # (batch_size, input_dim)
        optimizer.output = self.grad

        dW = (dLoss.T @ self.cache['X'].T) / optimizer.batch_size #
(output_dim, input_dim)
        dbias = np.sum(dLoss.T, axis=1, keepdims=True) /
optimizer.batch_size # (output_dim, 1)

        dW_weight_decay = self.W * optimizer.weight_decay /
optimizer.batch_size

        if need_update:
            self.W = self.W - (dW + dW_weight_decay) *
optimizer.learning_rate
            self.bias = self.bias - dbias.reshape(self.bias.shape) *
optimizer.learning_rate

        return dW + dW_weight_decay, dbias.reshape(self.bias.shape)
```

ZKC

We will train a GNN to cluster people in the karate club in such that people who are more likely to associate with either the officer or Mr. Hi will be close together, while the distance beween the 2 classes will be far.

In the original paper titled "Semi-Supervised Classification with Graph Convolutional Networks" that can be found here https://arxiv.org/pdf/1609.02907.pdf, the authors framed this as a node-level classification problem on a graph. We will pretend that we only know the affiliation labels for some of the nodes (which we'll call our training set) and we'll predict the affiliation labels for the rest of the nodes (our test set).

We will build a multi-layer Graph Convolutional Network (GCN) with the following layer-wise propagation rule:

$$H^{(l+1)} = \sigma\left(D^{-1/2}\widetilde{A}\,D^{-1/2}H^{(l)}W^{(l)}\right) = \sigma\left(\widetilde{A}^{SymNorm}H^{(l)}W^{(l)}\right)$$

where $\widetilde{A}$ is the adjacency matrix that we discussed in Homework 4, except that now we add the identity to include self loops at every node for numerical stability. D is the degree matrix as

defined in the past, $H^{(l)}$ is the activation matrix of the l-th layer, and W is the weight matrix to be learned. σ is an activation function, in this case tanh.

We used the python module networkx to import the dataset and provided some helper functions to help understand the data as shown below.

```
graph = ntwx.karate_club_graph()
num_nodes, num_edges, club_labels = get_graph_metadata(graph, 'club')
colors, labels, num_classes = get_colors_labels_and_classes(graph,
num_nodes)
Mr_Hi_people, Officer_people = get_affiliation(club_labels)
```

Data Inspection

```
print(f'ZKC dataset graph has {num_nodes} nodes and {num_edges}
edges')
```

```
ZKC dataset graph has 34 nodes and 78 edges
```

```
print(f'The affiation of each of the 34 members between the officer
and Mr. Hi is given below:\n {club_labels}')
```

```
The affiation of each of the 34 members between the officer and Mr. Hi
is given below:
 {0: 'Mr. Hi', 1: 'Mr. Hi', 2: 'Mr. Hi', 3: 'Mr. Hi', 4: 'Mr. Hi', 5:
'Mr. Hi', 6: 'Mr. Hi', 7: 'Mr. Hi', 8: 'Mr. Hi', 9: 'Officer', 10:
'Mr. Hi', 11: 'Mr. Hi', 12: 'Mr. Hi', 13: 'Mr. Hi', 14: 'Officer', 15:
'Officer', 16: 'Mr. Hi', 17: 'Mr. Hi', 18: 'Officer', 19: 'Mr. Hi',
20: 'Officer', 21: 'Mr. Hi', 22: 'Officer', 23: 'Officer', 24:
'Officer', 25: 'Officer', 26: 'Officer', 27: 'Officer', 28: 'Officer',
29: 'Officer', 30: 'Officer', 31: 'Officer', 32: 'Officer', 33:
'Officer'}
```

```
print(f'nodes/people loyal to Mr. Hi are:\n {Mr_Hi_people}')
```

```
nodes/people loyal to Mr. Hi are:
 [0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 16, 17, 19, 21]
```

```
print(f'nodes/people loyal to the officer are:\n {Officer_people}')
```

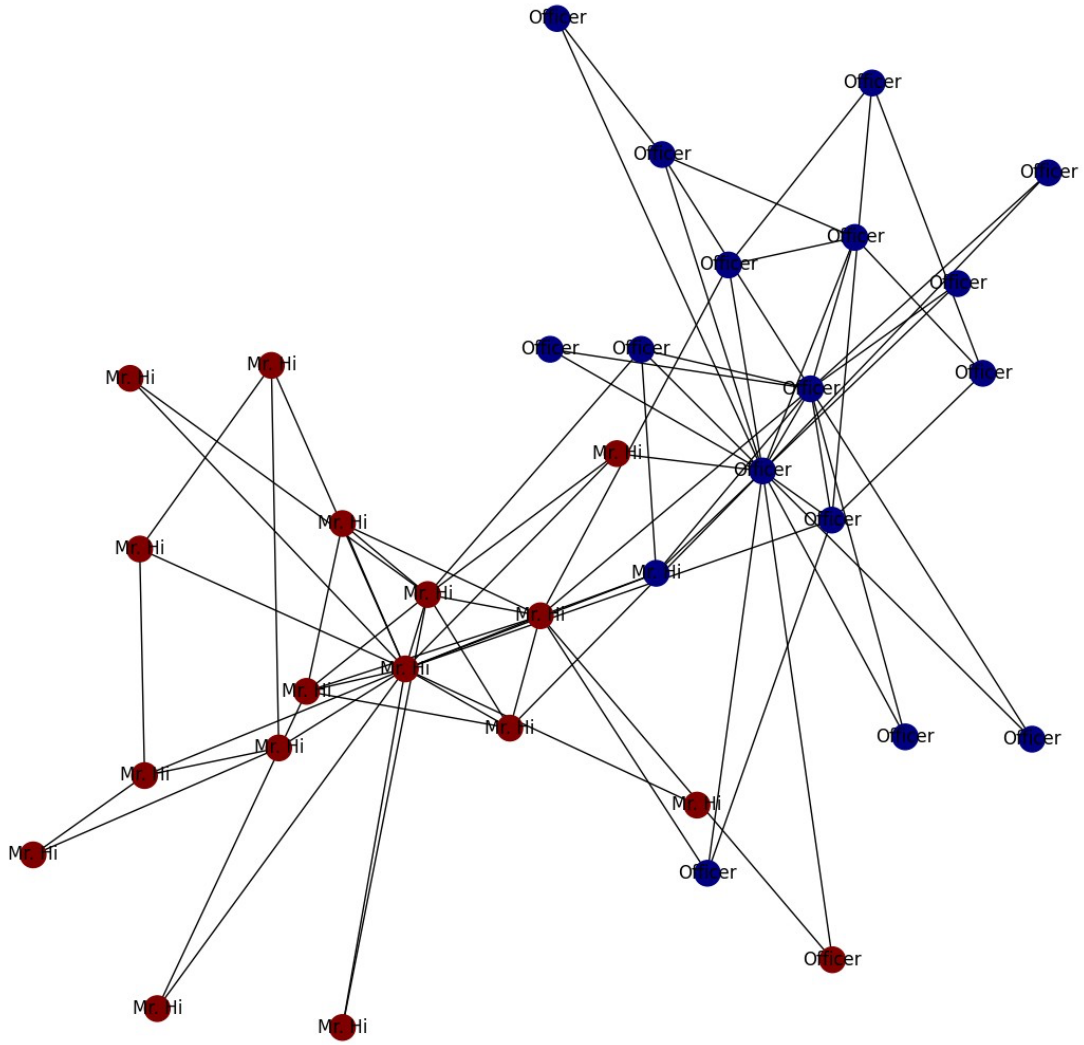
```
nodes/people loyal to the officer are:
 [9, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]
```

```
assert len(Officer_people) + len(Mr_Hi_people) == num_nodes
```

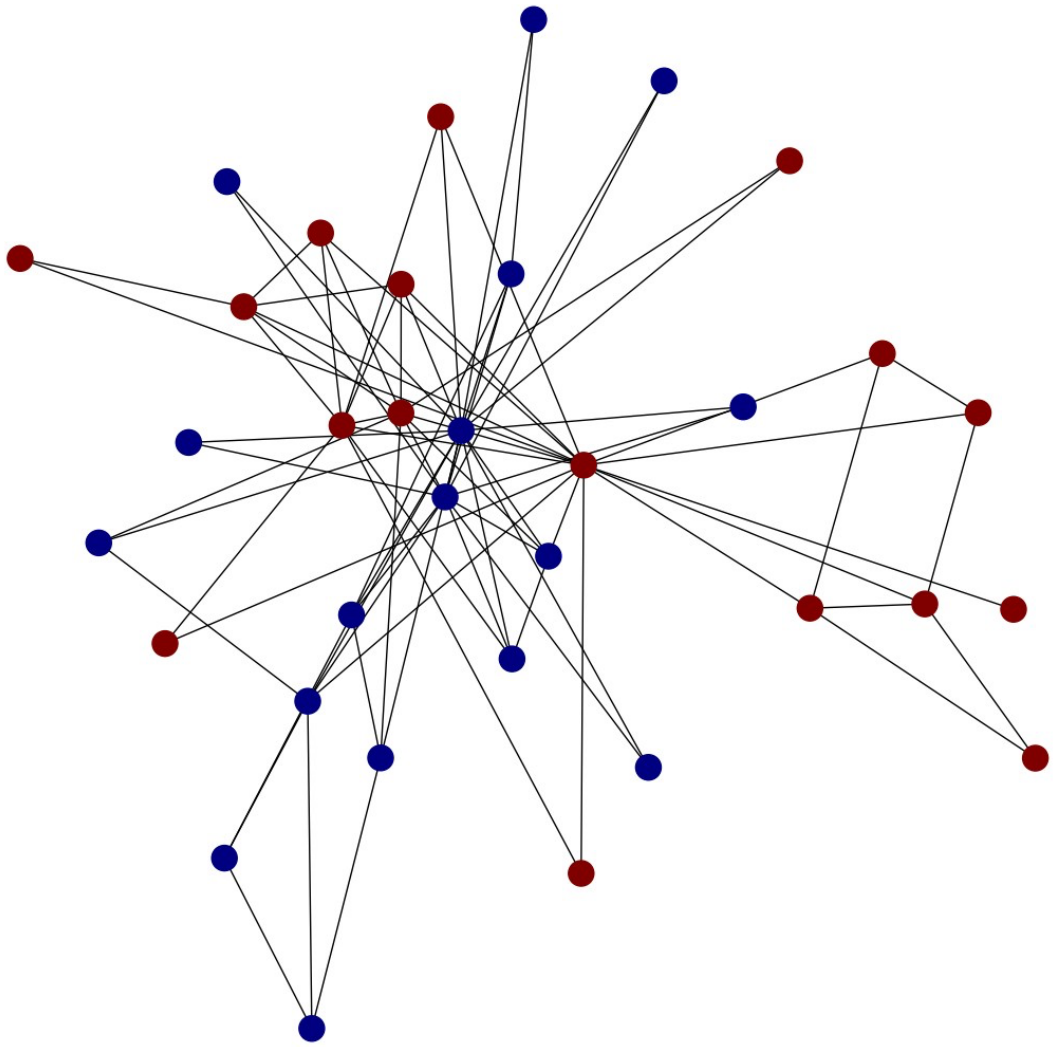
Note that the nodes represent the people and the edges represent the social interaction between the people outside on the club. Also the colors assigned to the nodes as shown below are indepedent of the class since no model has been trained to properly do that.

Show Graph with Labels

```
show_graph(graph=graph,
           label_values_of_nodes=club_labels,
           label_colors_of_nodes=colors,
           colors_of_edges='black',
           positions_of_nodes=None)
```


Officer
Officer
Officer

```
            colors_of_edges='black',
            positions_of_nodes=None,)
```



Note that node labels are obtained using the greedy modurality maximization algorithm descibted in the paper linked above.

Our goal in this problem is therefore to write a simple Graph Neural Network using python to perform node classification. We will also use the node embedding to move nodes with similar classes close to each other. We have provided here the adjacency matrix of the graph.

```
A = ntwx.to_numpy_array(graph)
A
```

```
array([[0., 4., 5., ..., 2., 0., 0.],
       [4., 0., 6., ..., 0., 0., 0.],
       [5., 6., 0., ..., 0., 2., 0.],
       ...,
       [2., 0., 0., ..., 0., 4., 4.],
       [0., 0., 2., ..., 4., 0., 5.],
       [0., 0., 0., ..., 4., 5., 0.]])
```

Note that we adjacency matrix does not include the node itself. We want our network to be aware of information about the nodes themselves instead of only the neighborhood, so we add self loops our adjacency matrix. The paper called this $\widetilde{A}$.

Q. 1. Compute: $\widetilde{A} = A_{self-loop} = A + I$

Where I is the identity matrix which allows us to include self loops of each nodes.

```
# TODO
# A_tild = ????
A_tild = A + np.eye(num_nodes)
```

Q.2. Write a function that takes in $\widetilde{A}$ as argument and returns the $\widetilde{A}^{SymNorm}$ adjacency matrix. You may find the provided function `fill_diagonal` useful, as well as the inverse function `np.linalg.inv` and the matrix square root function `np.linalg.sqrtm`.

```
def get_adjacency_matrix(A_tild):
    # TODO
    """Return symmetric normalized adjacency: D^{-1/2} A~ D^{-1/2}."""
    degrees = A_tild.sum(axis=1)
    inv_sqrt_deg = np.diag(1.0 / np.sqrt(degrees + 1e-12))
    return inv_sqrt_deg @ A_tild @ inv_sqrt_deg

A_hat = get_adjacency_matrix(A_tild)
A_hat
```

```
array([[0.02325581, 0.11136921, 0.13076645, ..., 0.06502561, 0.
,
        0.        ],
       [0.11136921, 0.03333333, 0.18786729, ..., 0.        , 0.
,
        0.        ],
       [0.13076645, 0.18786729, 0.02941176, ..., 0.        , 0.0549235
,
        0.        ],
       ...,
       [0.06502561, 0.        , 0.        , ..., 0.04545455,
0.13655775,
        0.12182898],
       [0.        , 0.        , 0.0549235 , ..., 0.13655775,
```

```
0.02564103,
        0.11437725],
       [0.        , 0.        , 0.        , ..., 0.12182898,
0.11437725,
        0.02040816]])
```

The other input to our GNN is the graph node matrix X which contains node features. For simplicity, we set X to be the identity matrix because we don't have any node features in this example. In a sense, this will map each node in the graph to a column of learnable parameters in the first layer, resulting in a fully learnable node embedings. In the question below, set the matrix X to be the identity.

Q.3. Generate the feature input matrix X

```
# TODO
# X = ??
X = np.eye(num_nodes)
```

Single GNN Layer Implementation

We will first implement a single layer GNN. Using the equation provided in the paper that is mentioned above, implement a forward and backward pass for a simple GNN layer.

Note that for l=0, H is the input X and $\widetilde{A}H$ does the message passing as we have seen in the previous homework and discussion, which is in turn multiplied by a weight matrix W. A non linearity is therefore applied afterward.

In the backward pass, we will apply L2 regularization to the weight matrix W. The regularization term is defined as: $\lambda \sum_{i,j} W_{i,j}^2$ which has gradient: $\lambda 2 W$. We will combine $\lambda 2$ in the weight decay parameter optimizer.weight_decay.

Q.4. Complete the #TODO to implement a forward pass that does just that in the class below.

```
class GNN_Layer():
    """process a single GNN layer"""
    def __init__(self, input_dim, output_dim, name=''):
        self.name = name
        self.cache = {}
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.W = get_xavier_init(self.output_dim, self.input_dim)
        self.activation = np.tanh

    def __repr__(self):
        dims = (self.input_dim, self.output_dim)
        if self.name:
            return f"GNN_Layer: W{'_' + self.name}{dims}"
        else:
            return f"GNN_Layer: W{'_'+ ''}{dims}"
```

```
    def forward_pass(self, A, X, W=None):
        """A here is the symmetricaly normalized adjacency matrix
        and X is the input to the layer. We cached some values
        to use in the backward pass."""

        self.cache['A'] = A # (batch_size, batch_size)
        # TODO
        # self.cache['X'] = ?? # (input_node_feature_dim, batch_size)
        self.cache['X'] = X  # (N, input_dim)

        # if W is None:
            # W = ??
        if W is None:
            W = self.W

        # H = ?? (hidden_dim, batch_size)
        H_linear = A @ self.cache['X'] @ W.T          # (N,
output_dim)
        # H = ??  (apply activation)
        H = self.activation(H_linear)                  # (N,
output_dim)
        # self.cache['H'] = ?? # (hidden_dim, batch_size)
        self.cache['H'] = H

        return H # (batch_size, hidden_dim)

    def backward_pass(self, optimizer, need_update=True):
        # derivative of tanh
        dtanh = 1.0 - self.cache['H'] ** 2            # (N,
output_dim)

        # gradient from next layer averaged over batch
        upstream = optimizer.output / optimizer.batch_size
        d = upstream * dtanh                           # (N,
output_dim)

        # gradient w.r.t. input features to pass to previous layer
        self.grad = self.cache['A'].T @ (d @ self.W)   # (N,
input_dim)
        optimizer.output = self.grad

        # gradient w.r.t. weights
        AX = self.cache['A'] @ self.cache['X']         # (N,
input_dim)
        dW = d.T @ AX                                   # (output_dim,
input_dim)

        # weight decay term
        dW_weight_decay = (optimizer.weight_decay /
```

```
optimizer.batch_size) * self.W

        if need_update:
            self.W = self.W - (dW + dW_weight_decay) *
optimizer.learning_rate

        return dW + dW_weight_decay
```

Since we are only trying to find out whether of a node belong to a class, we do not need to do message passing here. This function is provided in the helper functions above.

We now test the your implementation

lets's instantiate the GNN_Layer Class and the Softmax_Layer provided in the helper functions to test the gradients.

```
gnn_layer = GNN_Layer(input_dim=num_nodes,
                output_dim=2,
                name='gnn_layer_1')

sm_layer = Softmax_Layer(input_dim=2,
                  output_dim=num_classes,
                  name='softmax_layer')

optim = Grad_Descent_Optimizer(learning_rate=0,
                               weight_decay=1.)
```

Q. 5. lets compute the forward passes, uncomment and complete

```
# TODO
# gnn_layer_output = gnn_layer.forward_pass(A=??,
#                                           X=??)
gnn_layer_output = gnn_layer.forward_pass(A=A_hat, X=X)

# optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output),
y_true=labels)
optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)
```

lets verify that the layers are properly implemented by looking at the gradients. Note that need update is used only when we are updating the parameters during training. We do not need to do so here since we are just testing our layers.

```
dW_approx = compute_gradients(param_name="W",
                              layer=sm_layer,
                              inputs_data=(gnn_layer_output,),
                              gt_labels=labels,
                              eps=1e-4,
                              weight_decay=optim.weight_decay)

db_approx = compute_gradients(param_name="bias",
```

```
                                layer=sm_layer,
                                inputs_data=(gnn_layer_output,),
                                gt_labels=labels,
                                eps=1e-4,
                                weight_decay=optim.weight_decay)

dW, db = sm_layer.backward_pass(optim, need_update=False)
```

We then get the gradients of Softmax layer.

```
dW, db = sm_layer.backward_pass(optim, need_update=False)
```

We now assert that the true and approximate gradients are very close to each other. If not, something went wrong in our implementation.

```
implementation_check(dW, dW_approx, db, db_approx)
```

```
congrats, your implementation passes the test !!!
```

Q. 6. Now, use the GNN and Softmax layers implemented above to set up our GNN Network.

```
class GNN():
    """This class leverages the GNN layer implemented above by
cascading them."""
    def __init__(self,
                 input_dim,
                 output_dim,
                 hidden_dim,
                 num_layers,):

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.layers = []
        # first GNN layer
        first_gnn_layer = GNN_Layer(input_dim=input_dim,
                                    output_dim=hidden_dim[0],
                                    name='layer_0')
        self.layers.append(first_gnn_layer)

        # additional GNN layers (if any)
        for layer in range(1, num_layers):
            gnn_temp = GNN_Layer(input_dim=hidden_dim[layer-1],
                                 output_dim=hidden_dim[layer],
                                 name=f'layer_{layer}')
            self.layers.append(gnn_temp)
```

```
        # final softmax layer
        last_gnn_layer = Softmax_Layer(input_dim=hidden_dim[-1],
                                       output_dim=output_dim,
                                       name='sm_layer')
        self.layers.append(last_gnn_layer)

    def __repr__(self):
        return '\n'.join([str(layer) for layer in self.layers])

    def embedding(self, A, X):
        H = X
        for layer in self.layers[:-1]:
            H = layer.forward_pass(A, H)
        return H

    def forward_pass(self, A, X):
        H = self.embedding(A, X)
        out = self.layers[-1].forward_pass(H)
        return out
```

Q.7. uncomment and complete

```
# TODO
# gnn_model = GNN(
#     input_dim=??,
#     output_dim=??,
#     num_layers=2,
#     hidden_dim=[16, 2],
# )
# gnn_model
gnn_model = GNN(
    input_dim=num_nodes,
    output_dim=num_classes,
    num_layers=2,
    hidden_dim=[16, 2],
)
```

Train/ Test split

We chose nodes 0, 1, 8, 3, 8, 15, 16, 20, 25, 28, and 30 t to be our test nodes and used all the remaining for training.

```
test_nodes = np.array([0, 1, 8, 3, 8, 15, 16, 20, 25, 28, 30])
train_nodes = train_test_split(test_nodes, labels)

print(f'The nodes that will be used for training are:\n
{train_nodes}')
```

```
The nodes that will be used for training are:
 [ 2  4  5  6  7  9 10 11 12 13 14 17 18 19 21 22 23 24 26 27 29 31 32
33]

print(f'The nodes that will be used for test/val are:\n {test_nodes}')

The nodes that will be used for test/val are:
 [ 0  1  8  3  8 15 16 20 25 28 30]
```

We now instantiate our optimizer with a learning rate and weight decay for training.

```
training_optim = Grad_Descent_Optimizer(learning_rate=2e-2,
weight_decay=2.5e-2)
```

Q.8 Complete the training loop function below. Note that the train loss and test loss are computed over a given set of nodes that is defined by our training and testing set.

```
def train(model,
          A_hat,
          X,
          y_true,
          nodes_to_be_used_for_training,
          nodes_to_be_used_for_testing,
          threshold_value=.5,
          ealy_stopping_steps=60,
          num_epochs=20000):
    """trains our gnn model"""

    accs = []
    training_losses = []
    testing_losses = []
    embeddings = []
    y_preds = []

    minimum_loss = 1e7
    ealy_stopping_counter = 0

    for epoch in range(num_epochs):
        # forward
        y_pred = model.forward_pass(A_hat, X)
        training_optim(y_pred, y_true, nodes_to_be_used_for_training)

        # backward (update all layers)
        for layer in reversed(model.layers):
            layer.backward_pass(training_optim, need_update=True)

        # bookkeeping
        embeddings.append(model.embedding(A_hat, X))
        y_preds.append(y_pred)
```

```
        acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true,
axis=1))[
            [i for i in range(y_true.shape[0]) if i not in
nodes_to_be_used_for_training]
        ]
        accs.append(np.mean(acc_temp))

        # compute losses
        losses_per_node = get_cross_entropy(y_pred, y_true)
        train_loss_temp =
np.mean(losses_per_node[nodes_to_be_used_for_training])
        test_loss_temp =
np.mean(losses_per_node[nodes_to_be_used_for_testing])

        training_losses.append(train_loss_temp)
        testing_losses.append(test_loss_temp)

        if test_loss_temp < minimum_loss:
            minimum_loss = test_loss_temp
            ealy_stopping_counter = 0
        else:
            ealy_stopping_counter += 1

        if ealy_stopping_counter > ealy_stopping_steps:
            print("Training Stopped due to Early stopping!")
            break

        if epoch % 100 == 0:
            print(f"epoch #: {epoch+1} | train Loss:
{train_loss_temp:.3f} | test Loss: {test_loss_temp:.3f}")

    training_losses = np.array(training_losses)
    testing_losses = np.array(testing_losses)
    y_preds = threshold(np.array(y_preds), threshold_value)
    return training_losses, testing_losses, accs, embeddings, y_preds

training_losses, testing_losses, accs, embeddings, preds =
train(model=gnn_model,
                                  A_hat=A_hat,
                                  X=X,
                                  y_true=labels,

nodes_to_be_used_for_training=train_nodes,

nodes_to_be_used_for_testing=test_nodes,
                                  ealy_stopping_steps=50,
                                  num_epochs=20000)
```

```
epoch #: 1 | train Loss: 0.691 | test Loss: 0.691
epoch #: 101 | train Loss: 0.690 | test Loss: 0.689
```

```
epoch #: 201 | train Loss: 0.689 | test Loss: 0.688
epoch #: 301 | train Loss: 0.688 | test Loss: 0.686
epoch #: 401 | train Loss: 0.686 | test Loss: 0.684
epoch #: 501 | train Loss: 0.683 | test Loss: 0.681
epoch #: 601 | train Loss: 0.680 | test Loss: 0.677
epoch #: 701 | train Loss: 0.675 | test Loss: 0.673
epoch #: 801 | train Loss: 0.670 | test Loss: 0.667
epoch #: 901 | train Loss: 0.662 | test Loss: 0.659
epoch #: 1001 | train Loss: 0.653 | test Loss: 0.649
epoch #: 1101 | train Loss: 0.641 | test Loss: 0.637
epoch #: 1201 | train Loss: 0.626 | test Loss: 0.621
epoch #: 1301 | train Loss: 0.607 | test Loss: 0.602
epoch #: 1401 | train Loss: 0.584 | test Loss: 0.579
epoch #: 1501 | train Loss: 0.558 | test Loss: 0.552
epoch #: 1601 | train Loss: 0.527 | test Loss: 0.521
epoch #: 1701 | train Loss: 0.492 | test Loss: 0.487
epoch #: 1801 | train Loss: 0.456 | test Loss: 0.451
epoch #: 1901 | train Loss: 0.419 | test Loss: 0.414
epoch #: 2001 | train Loss: 0.382 | test Loss: 0.378
epoch #: 2101 | train Loss: 0.347 | test Loss: 0.343
epoch #: 2201 | train Loss: 0.315 | test Loss: 0.312
epoch #: 2301 | train Loss: 0.286 | test Loss: 0.283
epoch #: 2401 | train Loss: 0.260 | test Loss: 0.258
epoch #: 2501 | train Loss: 0.237 | test Loss: 0.236
epoch #: 2601 | train Loss: 0.218 | test Loss: 0.217
epoch #: 2701 | train Loss: 0.200 | test Loss: 0.200
epoch #: 2801 | train Loss: 0.186 | test Loss: 0.186
epoch #: 2901 | train Loss: 0.173 | test Loss: 0.173
epoch #: 3001 | train Loss: 0.162 | test Loss: 0.163
epoch #: 3101 | train Loss: 0.152 | test Loss: 0.153
epoch #: 3201 | train Loss: 0.144 | test Loss: 0.145
epoch #: 3301 | train Loss: 0.136 | test Loss: 0.137
epoch #: 3401 | train Loss: 0.130 | test Loss: 0.131
epoch #: 3501 | train Loss: 0.124 | test Loss: 0.125
epoch #: 3601 | train Loss: 0.119 | test Loss: 0.120
epoch #: 3701 | train Loss: 0.114 | test Loss: 0.115
epoch #: 3801 | train Loss: 0.110 | test Loss: 0.111
epoch #: 3901 | train Loss: 0.106 | test Loss: 0.107
epoch #: 4001 | train Loss: 0.103 | test Loss: 0.104
epoch #: 4101 | train Loss: 0.100 | test Loss: 0.101
epoch #: 4201 | train Loss: 0.097 | test Loss: 0.098
epoch #: 4301 | train Loss: 0.094 | test Loss: 0.095
epoch #: 4401 | train Loss: 0.092 | test Loss: 0.093
epoch #: 4501 | train Loss: 0.090 | test Loss: 0.091
epoch #: 4601 | train Loss: 0.088 | test Loss: 0.088
epoch #: 4701 | train Loss: 0.086 | test Loss: 0.087
epoch #: 4801 | train Loss: 0.084 | test Loss: 0.085
epoch #: 4901 | train Loss: 0.082 | test Loss: 0.083
epoch #: 5001 | train Loss: 0.080 | test Loss: 0.082
```

```
epoch #: 5101 | train Loss: 0.079 | test Loss: 0.080
epoch #: 5201 | train Loss: 0.077 | test Loss: 0.079
epoch #: 5301 | train Loss: 0.076 | test Loss: 0.078
epoch #: 5401 | train Loss: 0.075 | test Loss: 0.076
epoch #: 5501 | train Loss: 0.073 | test Loss: 0.075
epoch #: 5601 | train Loss: 0.072 | test Loss: 0.074
epoch #: 5701 | train Loss: 0.071 | test Loss: 0.073
epoch #: 5801 | train Loss: 0.070 | test Loss: 0.072
epoch #: 5901 | train Loss: 0.069 | test Loss: 0.071
epoch #: 6001 | train Loss: 0.068 | test Loss: 0.071
epoch #: 6101 | train Loss: 0.067 | test Loss: 0.070
epoch #: 6201 | train Loss: 0.066 | test Loss: 0.069
epoch #: 6301 | train Loss: 0.065 | test Loss: 0.068
epoch #: 6401 | train Loss: 0.064 | test Loss: 0.067
epoch #: 6501 | train Loss: 0.063 | test Loss: 0.067
epoch #: 6601 | train Loss: 0.062 | test Loss: 0.066
epoch #: 6701 | train Loss: 0.061 | test Loss: 0.065
epoch #: 6801 | train Loss: 0.060 | test Loss: 0.065
epoch #: 6901 | train Loss: 0.060 | test Loss: 0.064
epoch #: 7001 | train Loss: 0.059 | test Loss: 0.063
epoch #: 7101 | train Loss: 0.058 | test Loss: 0.063
epoch #: 7201 | train Loss: 0.057 | test Loss: 0.062
epoch #: 7301 | train Loss: 0.057 | test Loss: 0.062
epoch #: 7401 | train Loss: 0.056 | test Loss: 0.061
epoch #: 7501 | train Loss: 0.055 | test Loss: 0.061
epoch #: 7601 | train Loss: 0.054 | test Loss: 0.060
epoch #: 7701 | train Loss: 0.054 | test Loss: 0.060
epoch #: 7801 | train Loss: 0.053 | test Loss: 0.059
epoch #: 7901 | train Loss: 0.052 | test Loss: 0.059
epoch #: 8001 | train Loss: 0.052 | test Loss: 0.058
epoch #: 8101 | train Loss: 0.051 | test Loss: 0.058
epoch #: 8201 | train Loss: 0.051 | test Loss: 0.057
epoch #: 8301 | train Loss: 0.050 | test Loss: 0.057
epoch #: 8401 | train Loss: 0.049 | test Loss: 0.056
epoch #: 8501 | train Loss: 0.049 | test Loss: 0.056
epoch #: 8601 | train Loss: 0.048 | test Loss: 0.055
epoch #: 8701 | train Loss: 0.048 | test Loss: 0.055
epoch #: 8801 | train Loss: 0.047 | test Loss: 0.055
epoch #: 8901 | train Loss: 0.047 | test Loss: 0.054
epoch #: 9001 | train Loss: 0.046 | test Loss: 0.054
epoch #: 9101 | train Loss: 0.046 | test Loss: 0.053
epoch #: 9201 | train Loss: 0.045 | test Loss: 0.053
epoch #: 9301 | train Loss: 0.045 | test Loss: 0.053
epoch #: 9401 | train Loss: 0.044 | test Loss: 0.052
epoch #: 9501 | train Loss: 0.044 | test Loss: 0.052
epoch #: 9601 | train Loss: 0.043 | test Loss: 0.051
epoch #: 9701 | train Loss: 0.043 | test Loss: 0.051
epoch #: 9801 | train Loss: 0.042 | test Loss: 0.051
epoch #: 9901 | train Loss: 0.042 | test Loss: 0.050
```

```
epoch #: 10001 | train Loss: 0.041 | test Loss: 0.050
epoch #: 10101 | train Loss: 0.041 | test Loss: 0.050
epoch #: 10201 | train Loss: 0.040 | test Loss: 0.049
epoch #: 10301 | train Loss: 0.040 | test Loss: 0.049
epoch #: 10401 | train Loss: 0.040 | test Loss: 0.049
epoch #: 10501 | train Loss: 0.039 | test Loss: 0.048
epoch #: 10601 | train Loss: 0.039 | test Loss: 0.048
epoch #: 10701 | train Loss: 0.038 | test Loss: 0.048
epoch #: 10801 | train Loss: 0.038 | test Loss: 0.047
epoch #: 10901 | train Loss: 0.038 | test Loss: 0.047
epoch #: 11001 | train Loss: 0.037 | test Loss: 0.047
epoch #: 11101 | train Loss: 0.037 | test Loss: 0.046
epoch #: 11201 | train Loss: 0.037 | test Loss: 0.046
epoch #: 11301 | train Loss: 0.036 | test Loss: 0.046
epoch #: 11401 | train Loss: 0.036 | test Loss: 0.045
epoch #: 11501 | train Loss: 0.035 | test Loss: 0.045
epoch #: 11601 | train Loss: 0.035 | test Loss: 0.045
epoch #: 11701 | train Loss: 0.035 | test Loss: 0.044
epoch #: 11801 | train Loss: 0.034 | test Loss: 0.044
epoch #: 11901 | train Loss: 0.034 | test Loss: 0.044
epoch #: 12001 | train Loss: 0.034 | test Loss: 0.043
epoch #: 12101 | train Loss: 0.034 | test Loss: 0.043
epoch #: 12201 | train Loss: 0.033 | test Loss: 0.043
epoch #: 12301 | train Loss: 0.033 | test Loss: 0.043
epoch #: 12401 | train Loss: 0.033 | test Loss: 0.042
epoch #: 12501 | train Loss: 0.032 | test Loss: 0.042
epoch #: 12601 | train Loss: 0.032 | test Loss: 0.042
epoch #: 12701 | train Loss: 0.032 | test Loss: 0.042
epoch #: 12801 | train Loss: 0.031 | test Loss: 0.041
epoch #: 12901 | train Loss: 0.031 | test Loss: 0.041
epoch #: 13001 | train Loss: 0.031 | test Loss: 0.041
epoch #: 13101 | train Loss: 0.031 | test Loss: 0.041
epoch #: 13201 | train Loss: 0.030 | test Loss: 0.040
epoch #: 13301 | train Loss: 0.030 | test Loss: 0.040
epoch #: 13401 | train Loss: 0.030 | test Loss: 0.040
epoch #: 13501 | train Loss: 0.030 | test Loss: 0.039
epoch #: 13601 | train Loss: 0.029 | test Loss: 0.039
epoch #: 13701 | train Loss: 0.029 | test Loss: 0.039
epoch #: 13801 | train Loss: 0.029 | test Loss: 0.039
epoch #: 13901 | train Loss: 0.029 | test Loss: 0.039
epoch #: 14001 | train Loss: 0.028 | test Loss: 0.038
epoch #: 14101 | train Loss: 0.028 | test Loss: 0.038
epoch #: 14201 | train Loss: 0.028 | test Loss: 0.038
epoch #: 14301 | train Loss: 0.028 | test Loss: 0.038
epoch #: 14401 | train Loss: 0.028 | test Loss: 0.037
epoch #: 14501 | train Loss: 0.027 | test Loss: 0.037
epoch #: 14601 | train Loss: 0.027 | test Loss: 0.037
epoch #: 14701 | train Loss: 0.027 | test Loss: 0.037
epoch #: 14801 | train Loss: 0.027 | test Loss: 0.037
```

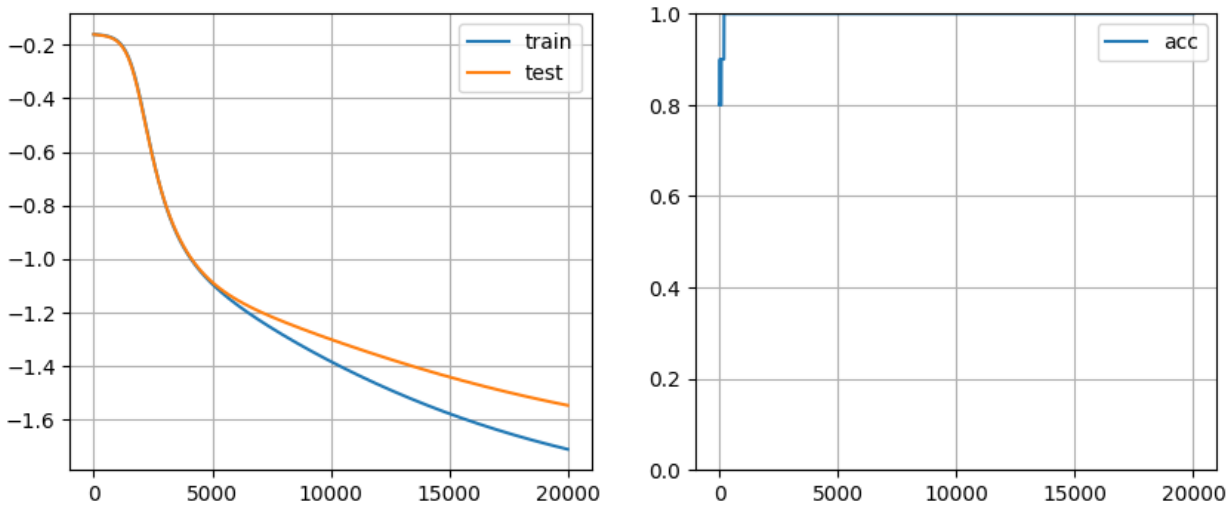
```
epoch #: 14901 | train Loss: 0.027 | test Loss: 0.036
epoch #: 15001 | train Loss: 0.026 | test Loss: 0.036
epoch #: 15101 | train Loss: 0.026 | test Loss: 0.036
epoch #: 15201 | train Loss: 0.026 | test Loss: 0.036
epoch #: 15301 | train Loss: 0.026 | test Loss: 0.036
epoch #: 15401 | train Loss: 0.026 | test Loss: 0.035
epoch #: 15501 | train Loss: 0.025 | test Loss: 0.035
epoch #: 15601 | train Loss: 0.025 | test Loss: 0.035
epoch #: 15701 | train Loss: 0.025 | test Loss: 0.035
epoch #: 15801 | train Loss: 0.025 | test Loss: 0.035
epoch #: 15901 | train Loss: 0.025 | test Loss: 0.034
epoch #: 16001 | train Loss: 0.025 | test Loss: 0.034
epoch #: 16101 | train Loss: 0.024 | test Loss: 0.034
epoch #: 16201 | train Loss: 0.024 | test Loss: 0.034
epoch #: 16301 | train Loss: 0.024 | test Loss: 0.034
epoch #: 16401 | train Loss: 0.024 | test Loss: 0.033
epoch #: 16501 | train Loss: 0.024 | test Loss: 0.033
epoch #: 16601 | train Loss: 0.024 | test Loss: 0.033
epoch #: 16701 | train Loss: 0.023 | test Loss: 0.033
epoch #: 16801 | train Loss: 0.023 | test Loss: 0.033
epoch #: 16901 | train Loss: 0.023 | test Loss: 0.033
epoch #: 17001 | train Loss: 0.023 | test Loss: 0.032
epoch #: 17101 | train Loss: 0.023 | test Loss: 0.032
epoch #: 17201 | train Loss: 0.023 | test Loss: 0.032
epoch #: 17301 | train Loss: 0.023 | test Loss: 0.032
epoch #: 17401 | train Loss: 0.022 | test Loss: 0.032
epoch #: 17501 | train Loss: 0.022 | test Loss: 0.032
epoch #: 17601 | train Loss: 0.022 | test Loss: 0.031
epoch #: 17701 | train Loss: 0.022 | test Loss: 0.031
epoch #: 17801 | train Loss: 0.022 | test Loss: 0.031
epoch #: 17901 | train Loss: 0.022 | test Loss: 0.031
epoch #: 18001 | train Loss: 0.022 | test Loss: 0.031
epoch #: 18101 | train Loss: 0.021 | test Loss: 0.031
epoch #: 18201 | train Loss: 0.021 | test Loss: 0.031
epoch #: 18301 | train Loss: 0.021 | test Loss: 0.030
epoch #: 18401 | train Loss: 0.021 | test Loss: 0.030
epoch #: 18501 | train Loss: 0.021 | test Loss: 0.030
epoch #: 18601 | train Loss: 0.021 | test Loss: 0.030
epoch #: 18701 | train Loss: 0.021 | test Loss: 0.030
epoch #: 18801 | train Loss: 0.021 | test Loss: 0.030
epoch #: 18901 | train Loss: 0.021 | test Loss: 0.030
epoch #: 19001 | train Loss: 0.020 | test Loss: 0.030
epoch #: 19101 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19201 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19301 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19401 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19501 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19601 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19701 | train Loss: 0.020 | test Loss: 0.029
```

```
epoch #: 19801 | train Loss: 0.020 | test Loss: 0.029
epoch #: 19901 | train Loss: 0.019 | test Loss: 0.028
```

```
print(f'For this toy example, the classification accuracy on the test set is {accs[-1]}')
```

```
For this toy example, the classification accuracy on the test set is 1.0
```

```
plot_training_curves(training_losses, testing_losses, accs)
```



Results

Let's observe the affiliation of people in our test nodes.

```
[club_labels[i] for i in test_nodes]
```

```
['Mr. Hi',
 'Mr. Hi',
 'Mr. Hi',
 'Mr. Hi',
 'Mr. Hi',
 'Officer',
 'Mr. Hi',
 'Officer',
 'Officer',
 'Officer',
 'Officer']
```

Let's observe the position where our trainined model predict them to be at.

```
embeddings[-1][test_nodes]
```

```
array([[-0.80037256, -0.26890226],
       [-0.68783243, -0.1960049 ],
       [ 0.36742907,  0.1293229 ],
       [-0.72516672, -0.23943924],
       [ 0.36742907,  0.1293229 ],
       [ 0.65240429,  0.21787388],
       [-0.58635296, -0.17355569],
       [ 0.64726194,  0.21843197],
       [ 0.85627181,  0.30720553],
       [ 0.38587701,  0.11309733],
       [ 0.50348194,  0.17048203]])
```

Let's observe the GT for values for the test nodes.

```
labels[test_nodes]
```

```
array([[0., 1.],
       [0., 1.],
       [1., 0.],
       [0., 1.],
       [1., 0.],
       [1., 0.],
       [0., 1.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.]])
```

Let's observe the predicted values for them.

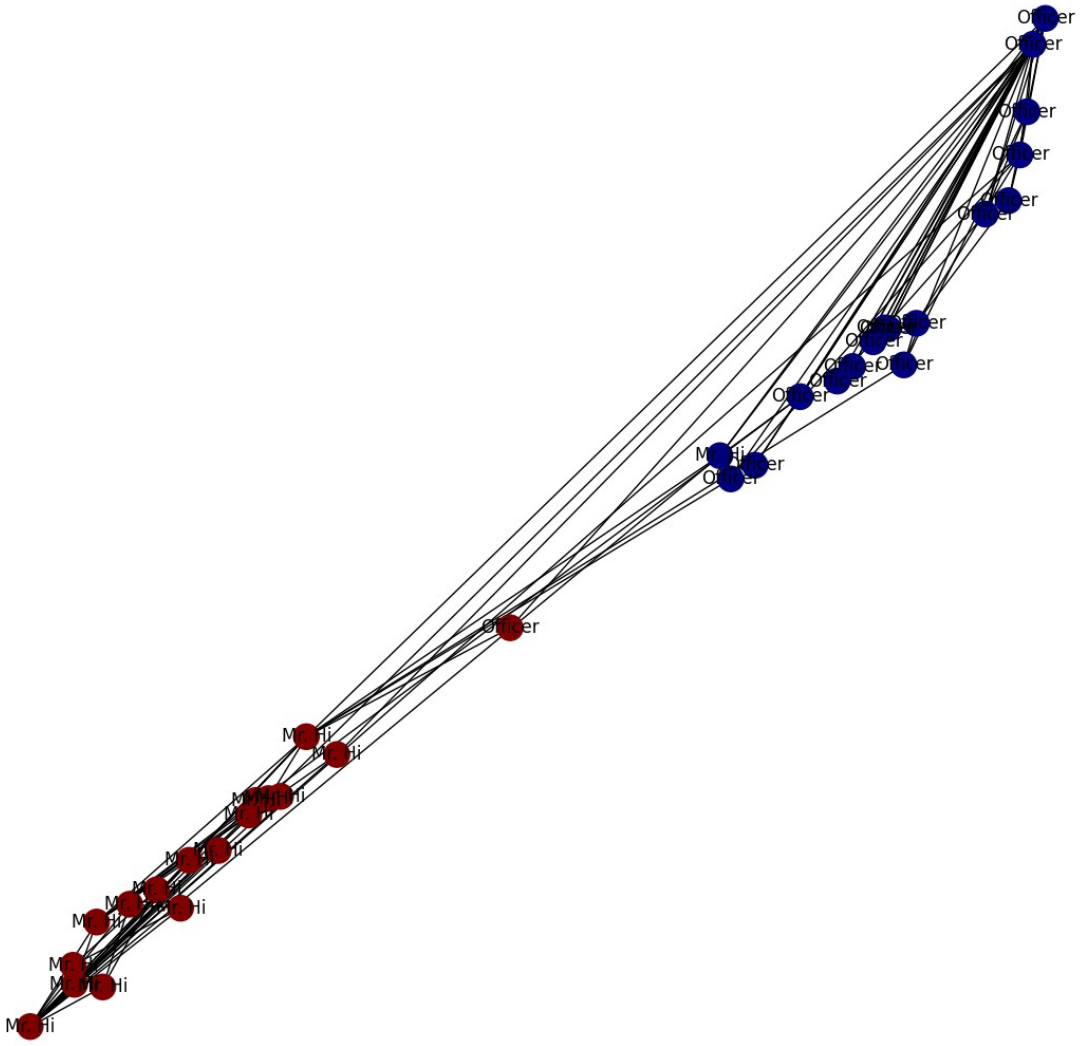
```
preds[-1][test_nodes]
```

```
array([[0., 1.],
       [0., 1.],
       [1., 0.],
       [0., 1.],
       [1., 0.],
       [1., 0.],
       [0., 1.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.]])
```

Finally, lets observe all the data clustered.

```
show_graph(graph=graph,
           label_values_of_nodes=club_labels,
           label_colors_of_nodes=colors,
```

```
            colors_of_edges='black',
            positions_of_nodes={i: embeddings[-1][i,:] \
                                for i in range(embeddings[-
1].shape[0])},
           )
```



Q. 9. Explain why we obtain a 100% on accuracy on our test set, yet we see in the plot above that 2 samples seem to be misclassified.

- The figure shows **embeddings**, not decision regions. The classifier is a **linear softmax** on the 2-D embedding, i.e., a hyperplane decision boundary. Two points can visually sit within the other cluster's "cloud" yet still lie on the correct side of the learned linear

boundary—thus correctly classified. High accuracy is therefore consistent with overlapping point clouds in 2D.

- The nodes are also colored by **ground-truth (community) labels** from greedy modularity, not predicted labels on that plot, so "misclassified" is a **visual** impression from overlap, not from the model's outputs. (The ground-truth labels themselves come from ZKC club split and NetworkX's `club` attribute.)
- Minor detail: the `test_nodes` list includes **node 8 twice**. That duplication does not change the correctness claim (both entries were predicted correctly), but it slightly weights node 8 more in the reported test accuracy.

All together: the model's linear boundary in embedding space classifies every test node correctly (100%), even though the 2-D embedding visualization shows some cross-cluster proximity/overlap—perfectly normal behavior.

6. Implementing Muon (Coding Question)

(a) Why scale to $\sqrt{3}$ (better than 1)?

For the cubic Newton–Schulz map $f(x) = \frac{3}{2}x - \frac{1}{2}x^3$, the fixed points are $x \in \{0, \pm 1\}$. When iterated, f converges to $\text{sign}(x)$ on the interval $[-\sqrt{3}, \sqrt{3}]$. Therefore, if every singular value σ of the matrix is $\leq \sqrt{3}$, repeated application drives $\sigma \to 1$. Pre-scaling to put the spectral/Frobenius norm at $\leq \sqrt{3}$ ensures all singular values start in that attraction region. Using "1" would be stricter (forces $\sigma \leq 1$) and can slow/over-shrink updates; $\sqrt{3}$ gives the largest safe radius for this cubic, allowing for faster progress per iteration while remaining in the convergence basin.

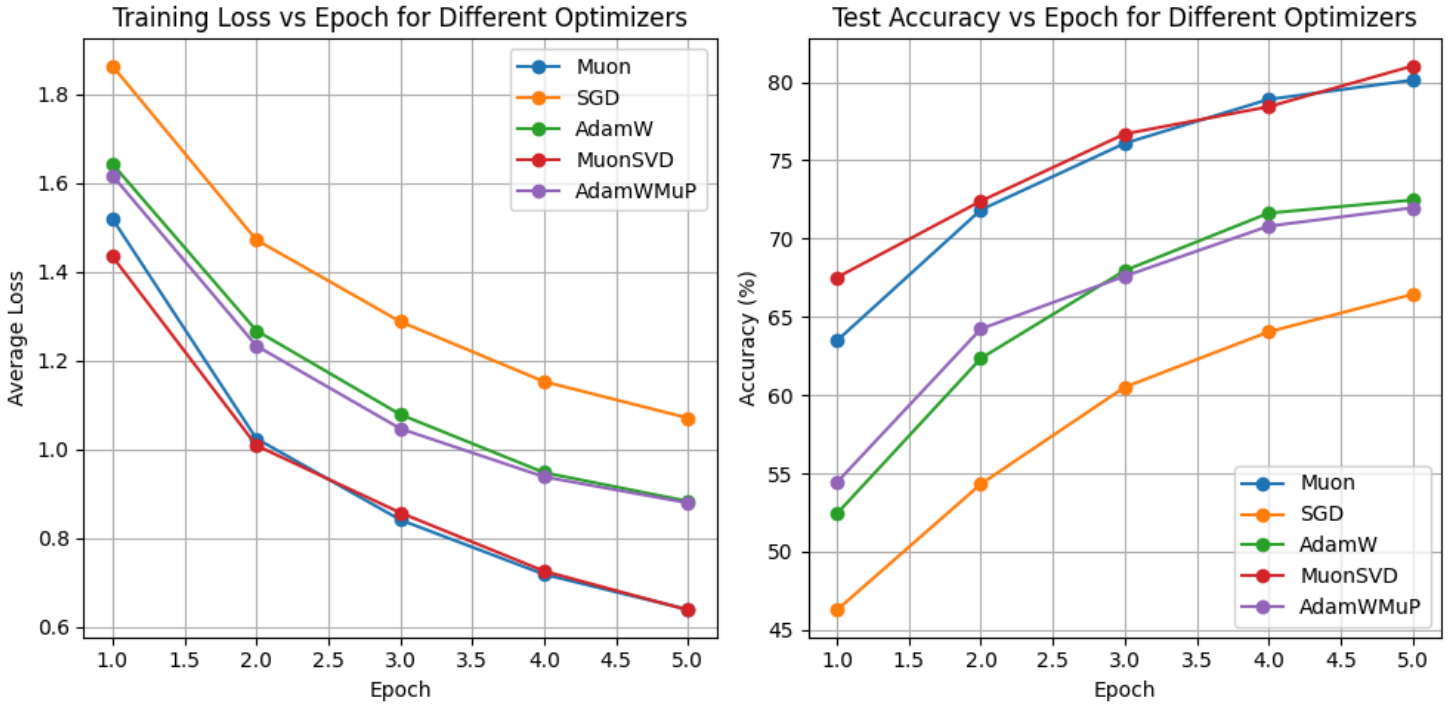
(b) How to reshape a conv kernel to $C' \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ so the layer is $C'x'$?

Using the usual **im2col/GEMM** lowering:

- Let the kernel $W \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times k \times k}$.
- Flatten the last three dims so $C' = \text{reshape}(W, [c_{\text{out}}, c_{\text{in}} \cdot k \cdot k])$. Here $d_{\text{out}} = c_{\text{out}}$ and $d_{\text{in}} = c_{\text{in}}k^2$.
- Form x' by extracting each $k \times k$ receptive field from the input (with stride/pad as configured), flattening to length $c_{\text{in}}k^2$, and stacking as columns.
- Then a forward pass is just matrix multiplication: $Y_{\text{col}} = C'x'$, followed by reshaping back to the output tensor.
  This is exactly the standard im2col formulation used to run convs as GEMMs.

(c) Which optimizer won (Muon vs SGD vs AdamW)?

From the provided run and figure, **Muon** achieved the best test accuracy across epochs (≈80.1% at epoch 5) and the lowest final loss; AdamW was second (~72.5%), and SGD third (~66.4%).



(d) Muon vs MuonSVD—are the curves expected?

Yes. MuonSVD performs *exact* orthogonalization by replacing M with $UV^\top$ from the SVD of M, so its loss/accuracy can be as good or slightly better than the approximate Newton–Schulz. The provided plot shows MuonSVD edging Muon slightly by the end. But SVD is substantially **slower** and less friendly on GPUs, while NS runs efficiently (even in bfloat16), which is why Muon uses NS: similar learning dynamics at much lower wall-clock cost. The logs show MuonSVD taking longer per epoch and even throwing an SVD convergence fallback warning—exactly in line with the intended design trade-off.

(e) (Optional) Faster Newton–Schulz (quintic) & speed vs SVD

A quintic NS step that converges faster than the cubic can be used. A common choice is
$f(x) = 3x - \frac{16}{5}x^3 + \frac{6}{5}x^5$,
applied in the same "polar-iteration" way: $X \leftarrow X(aI + bA + cA^2)$ with $A = X^\top X$. This brings singular values to 1 in fewer iterations (over roughly $[-\frac{3}{2}, \frac{3}{2}]$), improving wall-clock vs cubic NS. In practice, Muon(quintic, fewer iters) can retain MuonSVD-like losses while staying much faster than SVD because it is still just matmuls (no decompositions).

Sketch (drop-in)

```
def newton_schulz_orthogonalize_quintic(X, num_iters):
    dtype = X.dtype
    X = X.bfloat16()
    transposed = False
    if X.size(-2) < X.size(-1):
```

```
        transposed = True
        X = X.mT
    # scale to fall in the quintic's safe range
    X = X / (torch.linalg.norm(X, dim=(-2,-1), keepdim=True) + 1e-7) * (1.5)**0.5
    for _ in range(num_iters):
        A = X.mT @ X
        # f(x) = 3x - 16/5 x^3 + 6/5 x^5  ==>  X @ (aI + bA + cA^2)
        X = X @ (3*torch.eye(A.shape[0], dtype=X.dtype, device=X.device)
                 + (-16/5)*A + (6/5)*(A @ A))
    if transposed:
        X = X.mT
    return X.to(dtype)
```

Speed comment: Even with a quintic, Muon remains **much** faster than SVD-based orthogonalization because it is still only matrix–matrix multiplies (great for GPUs), whereas SVD is decomposition-heavy; this is exactly why Muon adopted NS over SVD.

(f) (Optional) Best hyperparameters found

From the sweep:

- **Muon:** best around $lr = 1e-2, weight_decay = 1e-3$ (≈76.7% validation).
- **AdamW:** best around $lr = 1e-3, weight_decay = 1e-4$ (≈68.1% validation).
  These choices align with μP's goal: keep update magnitudes consistent across widths/scales via appropriate scaling (the Muon implementation uses the μP-style $\sqrt{\max(1, d_{\text{out}}/d_{\text{in}})}$ factor), which helps stabilize behavior under different shapes.

---

7. Homework Process and Study Group

(a)

1. Lecture Notes
2. **A Gentle Introduction to Graph Neural Networks** (Kipf et al., 2021) - A foundational, highly-cited visual guide to GNN concepts, referenced extensively for its explanations of message passing, node/graph-level tasks, and analogies to CNNs.
3. **Semi-Supervised Classification with Graph Convolutional Networks** (Kipf & Welling, 2016) - The seminal paper that introduced Graph Convolutional Networks (GCNs), providing the core algorithm and normalization scheme for a key implementation assignment.
4. **Neural Message Passing for Quantum Chemistry** (Gilmer et al., 2017) - A key theoretical paper that formalized the Message Passing Neural Network (MPNN) framework, unifying many GNN models under a single conceptual umbrella.
5. **Deep Sets** (Zaheer et al., 2017) - Provides the essential theoretical justification for using permutation-invariant functions (sum, mean, max) to aggregate information from unordered neighbor sets in GNNs.
6. **NVIDIA System Management Interface (nvidia-smi)** (NVIDIA) - The official documentation for the `nvidia-smi` CLI, a critical tool used for monitoring GPU memory and answering questions about hardware specifications and CUDA context overhead.
7. **Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer** (Yang et al., 2022) - The paper introducing Maximal Update Parametrization (μP), the theoretical basis for the Muon optimizer's scaling strategy.
8. **Muon: An optimizer for hidden layers in neural networks** (Keller Jordan) - The primary source explaining the design of the Muon optimizer and its use of Newton-Schulz iteration as a fast, practical alternative to SVD for orthogonalization.
9. **Batch Normalization: Accelerating Deep Network Training** (Ioffe & Szegedy, 2015) - A landmark paper introducing a fundamental technique for training deep neural networks, referenced in the analysis of GNN architectural patterns.
10. **Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour** (Goyal et al., 2017) - An influential paper on training with large batch sizes, providing the context for the assignment exploring the trade-offs between batch size, memory, and model convergence.
11. **Newton-Schulz for Orthogonalization** (Modula Systems) - A technical document detailing the Newton-Schulz algorithm, the specific numerical method used by the Muon optimizer to efficiently approximate matrix orthogonalization.
12. **NetworkX: Karate Club Graph** (NetworkX Developers) - Official documentation for the Zachary's Karate Club graph, the classic dataset used for the hands-on GCN node classification task.
13. **The CIFAR-10 and CIFAR-100 datasets** (Krizhevsky, Nair, Hinton) - The homepage for the CIFAR-10 dataset, the standard benchmark used for image classification and GPU memory profiling in the first exercise.
14. **Graph machine learning with missing node features** (Twitter Engineering, 2022) - A practical guide on a common real-world problem, explaining how GNNs can successfully operate on graphs with incomplete data.
15. **PyTorch Forums: Optimizers memory usage** (PyTorch Community) - A practical community discussion that directly explains why optimizers like Adam consume more memory than SGD, answering a key question in the memory profiling assignment.
16. **Anatomy of a High-Speed Convolution** (Manas Sahni) - A clear explanation of the `im2col` technique, which reshapes convolution kernels into matrices to leverage highly optimized matrix multiplication routines.

(b)

None

(c)

2 days.. 💀